

Syncing Local State with a Prop

Q49

SVELTE 5 RUNES \$STATE \$DERIVED PROPS

INTERVIEW QUESTION

Question. How do you initialize a reactive local variable that dynamically syncs with an incoming prop using Runes?

ANSWER.

The two patterns — pick by whether the local copy must be editable

- ****Editable local copy** → `$state(prop)` ****** declare `let local = $state(prop.value)` to create a **deeply reactive local variable** (see Q34) whose initial value is copied from the prop. The user can then mutate `local` freely — typing into a bound input, toggling, etc.
- ****Read-only mirror** → `$derived(prop.value)` ****** declare `let mirror = $derived(prop.value)` to create a **memoized computed value** (see Q37) that re-evaluates whenever the prop changes. You cannot assign to a `$derived` — it is a pure function of its inputs.

****Why `$state(prop)` alone does not sync****

- **One-shot initialization:** writing `let local = $state(prop.value)` copies the prop's value **once**, at component setup. The `$state(...)` call runs a single time, so later changes to the prop do **not** flow into `local`.
- **The prop is a moving target:** props in Svelte 5 are reactive — the parent can pass a new value at any time, and the child sees the update — but `$state(initialValue)` only reads `initialValue` at birth.

- **The fix when you need both:** to get an editable local copy that *also* re-syncs when the prop changes, combine the two — declare `let local = $state(prop.value)` and add a `$effect(() => { local = prop.value; })` that watches the prop and overwrites the local copy on change (with care; see Q21 about avoiding effect loops).

The prescriptive rule

- ****Default to `$derived`: if the local value is a pure transform of the prop (formatting, filtering, computing), reach for `$derived`**** — it is always correct, never goes stale, and costs nothing when nobody reads it.
- ****Reach for `$state(prop)` only when the child must own its own edits**** to the value (form inputs, draft text, in-progress edits). Accept that you then own the synchronization back to the prop.
- ****Avoid the legacy `let x = prop`**** it neither tracks the prop nor participates in reactivity — a silent source of stale-UI bugs.

Editor.svelte

```
<script>
  let { initialText = '' } = $props();

  // Editable local copy – initialized once from the prop.
  let draft = $state(initialText);

  // Read-only mirror – recomputes whenever initialText
  changes.
  let preview = $derived(draft.toUpperCase());
</script>

<input bind:value={draft} />
<p>Preview: {preview}</p>
```

- ① **Summary.** Use `$state(prop.value)` for an editable local copy initialized once from a prop, and `$derived(prop.value)` for a read-only computed mirror that re-syncs automatically. `$state(prop)` alone does not track later prop changes — pair it with a `$effect` only when you need both editability and continuous sync.

